Available online at www.sciencedirect.com

ScienceDirect

IFAC Papers Online CONFERENCE PAPER ARCHIVE

IFAC PapersOnLine 56-2 (2023) 8085–8090

A Novel Multi-dimensional Time-series Data Anomaly Detection Model Based on Generative Adversarial Network Aided Autoencoder

Zejun Gong * **Qiang Liu** * **Jinliang Ding** * **Xiaobo Wang** * **Peng Wang** **

* *State Key Laboratory of Synthetical Automation for Process Industries, Northeastern University, Shenyang, China*
** *School of Computer Science, Fudan University, Shanghai, China*
2102024@stu.neu.edu.cn; liuq@mail.neu.edu.cn; jlding@mail.neu.edu.cn; 13816345617@139.com; pengwang5@fudan.edu.cn

Abstract: With increasing amount and easiness of access the data in industrial processes, data-driven technologies have become more prevalent in process monitoring. Anomaly detection is an indispensable part of process monitoring. However, most industrial data are closely related to time, and classical anomaly detection algorithms mostly focus on learning the features of static data, ignoring the dynamic features of industrial data. This paper proposes a multi-dimensional time-series data anomaly detection model based on generative adversarial network aided autoencoder. By extracting the features of the normal time-series data, feature representation is established in latent space. Meanwhile, we introduce generated adversarial network (GAN) into the autoencoder (AE) training to enhance the feature learning ability of the autoencoder, so that the normal time-series data can be well represented in the latent space. Gated recurrent unit (GRU) is used as the main network of the autoencoder to learn the dynamic features between different time steps in the sequence data and detect fault data through the value of the reconstruction error. We verify the validity of the proposed model in simulation data and apply it to the real anomaly detection of steel plate production. Compared with k-nearest neighbor, linear discriminant analysis, principal component analysis, one-class support vector machine, data-enhanced method and the traditional dynamic autoencoder, the proposed method performs the best.

Copyright © 2023 The Authors. This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0/)

Keywords: anomaly detection,multi-dimensional time series,time-series generation adversarial network,autoencoder

1. INTRODUCTION

With data in industrial processes being larger in scale and easier to access, data-driven technologies become more and more important in process monitoring (Yin et al. (2014)). Moreover, due to the development of modern computer and database technologies, machine learning has become a popular technology in data-driven process monitoring. As an important task in process monitoring, the accurate detection of fault conditions in the production process is essential to healthy operation of the whole production line and quality improvement of the products.

Recently, many data-driven anomaly detection methods have been proposed. They can be divided into supervised and unsupervised algorithms. Supervised algorithms such as k-nearest neighbor (KNN) (Abeywickrama et al. (2016)), linear discriminant analysis (LDA) (Zhu et al. (2022)) and support vector machines (SVM) (Tang (2013)) perform well on sufficient data. However, in the practical industrial processes, most production lines operate under healthy conditions. It is difficult to collect fault data. The number of healthy data is often much more than fault data, which greatly affects the classification accuracy of supervised anomaly detection algorithm. Unsupervised anomaly detection algorithms usually build models by learning features of healthy samples (majority class), which can avoid the above problem. Principal component analysis (PCA) (Shlens (2014)), local outlier factor (LOF) (Cheng et al. (2019)), one-class SVM (OCSVM) (Schölkopf et al. (2001)) determine the boundary or control limit by learning the distribution of majority class samples to detect fault samples. However, most industrial process data is closely related to time. There is a strong time dependence between two time steps. Most unsupervised algorithms focus on learning static features of data, but ignore the dynamic features among data.

Since deep neural network has good feature representation ability and strong generalization, deep learning method has become a research hotspot. Tian et al. (2022) pro-

⋆ Corresponding author: Qiang Liu

2405-8963 Copyright © 2023 The Authors. This is an open access article under the CC BY-NC-ND license.
Peer review under responsibility of International Federation of Automatic Control.
10.1016/j.ifacol.2023.10.960

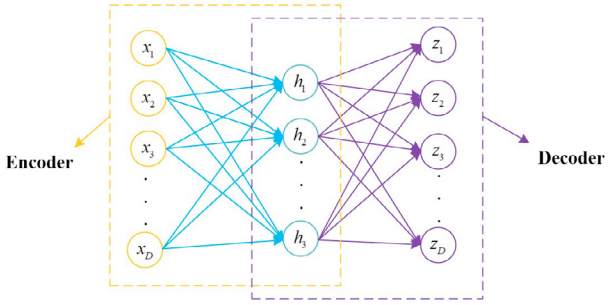


Fig. 1. The structure of an autoencoder.

posed the dynamic autoencoder (AE) model, which can learn the dynamic features between data well. However, the intricacy of its network architecture may give rise to overfitting, thereby potentially compromising the model's ability to generalize. Yoon et al. (2019) proposed adversarial learning strategy of time-series generative adversarial network (TimeGAN) to learn the dynamic and static features of fault samples. TimeGAN generates fault data to solve the problem of data imbalance. When only a few fault samples are available, insufficient samples cannot support TimeGAN training, leading to mode collapse. Although many anomaly detection algorithms have been proposed, existing methods still have some limitation on extracting dynamic features.

In this paper, a multi-dimensional time series anomaly detection model based on generative adversarial network aided autoencoder is proposed. By learning the features of normal time series data, a feature representation space is established in latent space so that the normal time series data can be well represented in latent space. The anomaly detection is completed according to the reconstruction error between the original data and the reconstructed data. Then the proposed method is compared with supervised algorithms, unsupervised algorithms and anomaly detection models based on data enhancement.

The layout of the paper is as follows. Section 2 provides a complete review of autoencoder, generative adversarial network, and gated recurrent unit. Section 3 focuses on the multi-dimensional time series anomaly detection model based on generative adversarial network aided autoencoder. Subsequently, in Section 4, the proposed method is tested on simulated data and real industrial data. Comparisons between the proposed method and the statistical learning methods, the deep learning methods are presented in Section 4, followed by a concluding remark in Section 5.

2. PRELIMINARIES

2.1 Autoencoder

An autoencoder consists of two parts, an encoder and a decoder, shown in fig 1. It learns the encoding of an input dataset (latent representation) in an unsupervised fashion. The encoder maps the input data from a high-dimensional space into codes in a low-dimensional space, and the decoder reconstructs the input data from the corresponding codes (Zhang et al. (2019)).

Given the training samples $\boldsymbol{x} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, ...\boldsymbol{x}_m\}$ (for each sample $\boldsymbol{x}_i$, $\boldsymbol{x}_i = [x_1, x_2, ...x_D]^T$), the encoder transforms the input vector $\boldsymbol{x}$ into a hidden representation $\boldsymbol{h} = \{\boldsymbol{h}_1, \boldsymbol{h}_2, ...\boldsymbol{h}_m\}$ (for each sample $\boldsymbol{h}_i$, $\boldsymbol{h}_i = [h_1, h_2, ...h_d]^T$) through the sigmoid function as follows

$$\boldsymbol{h} = s_f(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}) \tag{1}$$

$$s_f(t) = 1/(1 + e^{-t}) \tag{2}$$

where $\boldsymbol{x}$ and $\boldsymbol{h}$ are D-dimensional and d-dimensional vectors, respectively, $\boldsymbol{W}^{(1)}$ is a $d \times D$-dimensional weight matrix and $\boldsymbol{b}^{(1)}$ is a d-dimensional bias vector.

Then, the vector $\boldsymbol{h}$ is transformed back to a reconstruction vector$\boldsymbol{z} = \{\boldsymbol{z}_1, \boldsymbol{z}_2, ...\boldsymbol{z}_m\}$ (for each sample $\boldsymbol{z}_i$, $\boldsymbol{z}_i = [z_1, z_2, ...z_D]^T$) by the decoder as follows

$$\boldsymbol{z} = s_f(\boldsymbol{W}^{(2)}\boldsymbol{x} + \boldsymbol{b}^{(2)}) \tag{3}$$

where $\boldsymbol{z}$ is a D-dimensional vector, $\boldsymbol{W}^{(2)}$ is a $D \times d$-dimensional weight matrix and $\boldsymbol{b}^{(2)}$ is a D-dimensional bias vector.

The autoencoder training aims to optimize the parameter set $\boldsymbol{\theta} = \{\boldsymbol{W}^{(1)}, \boldsymbol{b}^{(1)}, \boldsymbol{W}^{(2)}\boldsymbol{b}^{(2)}\}$ minimizing the reconstruction error. The mean square error(MSE) is usually used as the standard autoencoder loss function

$$J_{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} L_{MSE}(\boldsymbol{x}_i, \boldsymbol{z}_i) = \frac{1}{m}\sum_{i=1}^{m}(\frac{1}{2}\|\boldsymbol{z}_i - \boldsymbol{x}_i\|^2) \tag{4}$$

2.2 Generative Adversarial Networks

GAN is a deep generative model proposed by (Goodfellow et al. (2014)) in 2014 for unsupervised and semi-supervised learning. It consists of two networks, one for generating realistic data, called generator, and the other is to evaluate the authenticity of the samples, called discriminator. The two compete against each other, the generator is trained to produce samples that can fool the discriminator, and the discriminator is trained to discern the samples coming from the generator (fake samples) and the samples coming from theoriginal dataset (real samples) as much as possible. In theory, the final well-trained generator can generate extremely realistic samples, making the discriminator completely confusing and unable to distinguish real or fake samples.

The generator G learns a mapping from noise space to sample space, $G : Z \rightarrow X$, in order to capture the real data distribution and generate the realistic samples $G(z) \in X$. The role of the discriminator D is to give the probability that the sample comes from the real data space, $D : X \rightarrow [0, 1]$.

The two perform a zero-sum game, which means a minimax optimization. The value function $V(D, G)$ of generator and discriminator is as follows:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))] \tag{5}$$

where $x \in X$ is sampled from the real data distribution $p_{data}(x)$, $z \in Z$ is sampled from the Gaussian random

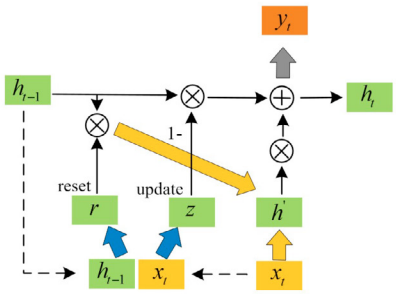


Fig. 2. The structure of GRU.

distribution $p_z(z)$. The alternate training of generator network and discriminator network composes the overall training process, and both of them are trained to achieve optimality through gradient descent algorithm.

2.3 Gated Recurrent Unit

The gated recurrent unit (GRU) is unit of recurrent neural networks (RNNs) where the current output is fed-back to the network and used as an input in future steps combined with other past inputs, similar to the idea of autoregressive dynamic models (Bansal et al. (2016)). However, RNNs will have vanishing/exploding gradients due to too deep network and other reasons, so it is difficult to train RNNs using stochastic gradient descent (SGD) algorithm. This problem is solved in the GRU network by making architectural variations to the RNNs. Thus the update of the unit values of the standard GRU is updated based on two gates as follows: The reset gate r selects information from the previous state h_{t-1} to be discarded. The update gate z selects new information from the input vector $\boldsymbol{x}_t$ and the previous state h_{t-1} to be added to a new state.

The use of these memory gates allows the GRU network, shown schematically in Fig 2, to selectively add or remove-information over long period of time and extract dynamic features.

3. A MULTI-DIMENSIONAL TIME SERIES ANOMALY DETECTION MODEL BASED ON GENERATIVE ADVERSARIAL NETWORK AIDED AUTOENCODER

The proposed model consists of three network components: embedding function, recovery function and sequence discriminator. While embedding network and recovery network is composed of autoencoder, the embedding network and sequence discriminator can be regarded as a generation adversarial network. The autoencoder is trained jointly with the adversarial components to make the embedding network to obtain excellent feature extraction ability. In this way, the proposed model can simultaneously learn to encode features, generate representations, and iterate across time. The network structure of the proposed model is is shown in Fig 3.

3.1 Embedding and Recovery Functions

The embedding function and recovery function can be regarded as an autoencoder, which has a good feature

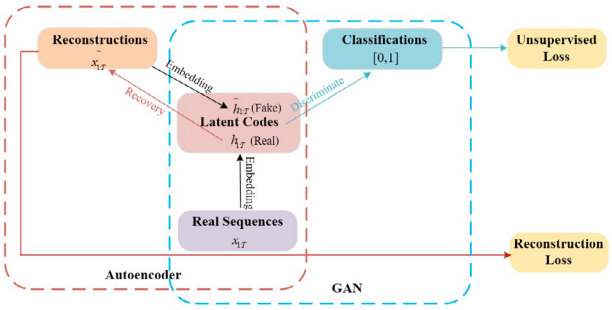


Fig. 3. The network structure of the proposed model.

representation of normal data in the latent space, and finally detects anomalies through reconstruction errors. In this paper, we implement e via a recurrent network,

$$\boldsymbol{h}_t = e_{\boldsymbol{x}}(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t) \tag{6}$$

where $e_{\boldsymbol{x}}$ is a recurrent embedding network for temporal features, $\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, ..., \boldsymbol{x}_T)$ $(t \in [1, 2, ..., T]$, for each sample $\boldsymbol{x}_t$, $\boldsymbol{x}_t = [x_1, x_2, ..., x_D]^T)$ is a time-series data containing T time steps, $\boldsymbol{h}_{1:T} = (\boldsymbol{h}_1, ..., \boldsymbol{h}_T)$ (for each sample $\boldsymbol{h}_t$, $\boldsymbol{h}_t = [h_1, h_2, ..., h_d]^T$) is latent variable of $\boldsymbol{x}_{1:T}$. In the opposite direction, we implement r through a feedforward network at each step,

$$\tilde{\boldsymbol{x}}_t = r_{\boldsymbol{x}}(\boldsymbol{h}_t) \tag{7}$$

where r_x is a recovery networks for temporal embeddings, $\tilde{\boldsymbol{x}}_{1:T} = (\tilde{\boldsymbol{x}}_1, \tilde{\boldsymbol{x}}_2, ..., \tilde{\boldsymbol{x}}_T)$ $(t \in [1, 2, ..., T]$, for each sample $\tilde{\boldsymbol{x}}_t$, $\tilde{\boldsymbol{x}}_t = [\tilde{x}_1, \tilde{x}_2, ..., \tilde{x}_D]^T)$ is a time-series data recovered from $\boldsymbol{h}_{1:T}$. The embedding and recovery function must be autoregressive and obey causal ordering (output at each step can only depend on preceding information). Here, we choose GRU as the main structure of the embedding and recovery network.

As a reversible mapping between the feature space and the latent space, the embedding function and the recovery function should be able to map the original time-series data $\boldsymbol{x}_{1:T}$ to the latent space $\boldsymbol{h}_{1:T}$ and accurately reconstruct the original data $\tilde{\boldsymbol{x}}_{1:T}$ from the latent space. So our first objective function is the reconstruction loss.

$$L_R = \mathbb{E}_{\boldsymbol{x}_{1:T} \sim p}\left[\sum_t \|\boldsymbol{x}_t - \tilde{\boldsymbol{x}}_t\|_2\right] \tag{8}$$

3.2 Embedding Fuction and Sequence Discriminator

Instead of the standard generative adversarial network, we consider the embedding network in the autoencoder as generative network. The input to the generated network in the proposed model is no longer noise, but the output $\tilde{\boldsymbol{x}}_{1:T}$ through the autoencoder. The latent variable $\boldsymbol{h}_{1:T}$ output by the embedding function for the first time is regarded as real data. The latent variable output of $\tilde{\boldsymbol{x}}_{1:T}$ through the embedding function is $\tilde{\boldsymbol{h}}_{1:T}$ as synthetic data. The autoregressive embedding function receives both the input from the original space $\tilde{\boldsymbol{x}}_t$ and its own output in the latent space $\tilde{h}_{t-1}$ to generate the next synthetic vector.

$$\tilde{\boldsymbol{h}}_t = e_{\boldsymbol{x}}(\tilde{\boldsymbol{h}}_{t-1}, \tilde{\boldsymbol{x}}_t) \tag{9}$$

$$\hat{y_t} = d_{\boldsymbol{x}}(\hat{\boldsymbol{h}_t}, \hat{y}_{t-1}) \tag{10}$$

We implement d through a GRU network, Where $d_{\boldsymbol{x}}$ is a classification functions, $\hat{\boldsymbol{h}}_t$ notation denotes either real ($\boldsymbol{h}_t$) or synthetic ($\tilde{\boldsymbol{h}}_t$) embeddings and $\hat{y_t}$ notation denotes classifications of either real (y_t) or synthetic ($\tilde{y}_t$) data. Compared to LSTM, GRUs have fewer parameters and train faster. The network allows maximizing (for the discriminator) or minimizing (for the generator) the likelihood of providing correct classifications $\hat{y_t}$ for both the training data $\boldsymbol{h}_t$ as well as for synthetic output $\tilde{\boldsymbol{h}_t}$ from the generator.

$$\begin{aligned} L_U = &\mathbb{E}_{\boldsymbol{x}_{1:T}\sim p}\left[\sum\nolimits_t \log d_{\boldsymbol{x}}(\boldsymbol{h}_t)\right] \\ &+ \mathbb{E}_{\boldsymbol{x}_{1:T}\sim p}\left[\sum\nolimits_t \log(1 - d_{\boldsymbol{x}}(e_{\boldsymbol{x}}(\tilde{\boldsymbol{x}}_t)))\right] \end{aligned} \tag{11}$$

3.3 Joint Training

Firstly, we pretrain the autoencoder so that the embedding and recovery network can acquire certain feature representation ability. Then the joint training of autoencoder and generated adversarial network is carried out. In the joint training, the time series $\boldsymbol{x}_{1:T}$ is mapped by the embedding network to $\boldsymbol{h}_{1:T}$ in the latent space, and $\boldsymbol{h}_{1:T}$ is reconstructed as time series $\tilde{\boldsymbol{x}}_{1:T}$ through the recovery network. $\tilde{\boldsymbol{x}}_{1:T}$ input embedding network to get the second encoding of $\boldsymbol{x}_{1:T}$ in the latent space mapping $\tilde{\boldsymbol{h}}_{1:T}$. The latent variable $\boldsymbol{h}_{1:T}$ encoded in the first time is considered to be the real sample, while the latent variable $\tilde{\boldsymbol{h}}_{1:T}$ encoded in the second time is considered to be the synthetic sample. $\boldsymbol{h}_{1:T}$, $\tilde{\boldsymbol{h}}_{1:T}$ is trained by input sequence discriminator. When Nash equilibrium is reached, the distributions of real sample distribution and the synthetic sample are basically close, that is, the autoencoder obtains a good feature representation.

3.4 Anomaly Detection

In anomaly detection, we will not use the discriminator, but detect fault samples through the value of reconstruction error. We introduce training data (normal samples) $\boldsymbol{x}_{1:T}$ into the autoencoder and get the reconstructed data $\tilde{\boldsymbol{x}}_{1:T}$ through the process of encoding and decoding. Then the reconstruction error is calculated by the following equation.

$$E_r = \frac{1}{T}\sum_{t=1}^{T} \|\boldsymbol{x}_t - \tilde{\boldsymbol{x}_t}\|_2 \tag{12}$$

The probability density of reconstruction error is calculated by kernel density estimation (KDE). The control limit of reconstruction error is determined according to the criterion that 95% of normal samples is lower than the control limit. The test samples are input into the autoencoder and their reconstruction errors are obtained. Because fault samples did not participate in the training process, their reconstruction error is high. If the reconstruction error is lower than the control limit, it is judged to be normal; if the reconstruction error is higher than the control limit, it is judged to be fault.

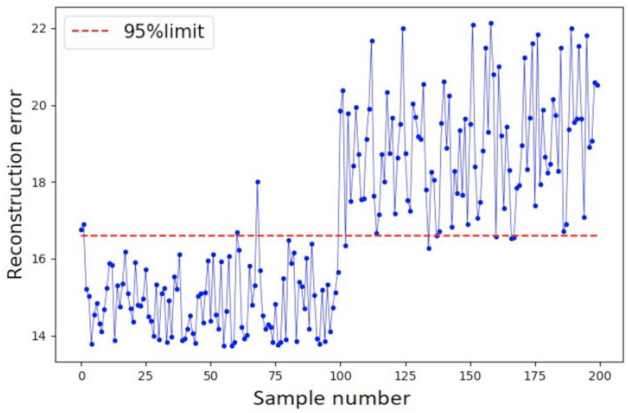


Fig. 4. The reconstruction error curve of simulation data.

4. CASE STUDIES

Two case studies are presented in this section. The effectiveness of the proposed multi-dimensional time series anomaly detection model based on generative adversarial network is proved.

4.1 Simulation Data

In this simulation, x_t is generated by a vector autoregressive model. Each normal simulation data contains 6 variables and 30 time steps.

$$\begin{aligned} &\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{A}\boldsymbol{x}_{t-1} + \boldsymbol{v}_t \\ &\boldsymbol{A} = \\ &\begin{pmatrix} -0.379 & 0.307 & 0.884 & -0.993 & 0.171 & 0.904 \\ 0.033 & 0.133 & -0.326 & -0.952 & -0.964 & 0.606 \\ -0.389 & -0.998 & 0.769 & 0.996 & 0.781 & -0.854 \\ -0.447 & -0.357 & -0.738 & -0.036 & -0.334 & 0.191 \\ -0.551 & -0.348 & 0.981 & 0.464 & 0.749 & 0.726 \\ -0.934 & 0.971 & -0.628 & 0.995 & 0.053 & 0.601 \end{pmatrix} \\ &\boldsymbol{c} = (\ 0.539\ \ -0.701\ \ 0.521\ \ -0.924\ \ 0.142\ \ 0.528\ )^T \end{aligned} \tag{13}$$

where $\boldsymbol{v}_t \in U(-1, 1)$,$A$ is a randomly generated 6x6 matrix. The first 2000 pieces of data are used for training the model, and the last 100 pieces of data are used as test data. To test the validity of anomaly detection of the proposed model, the following types of fault is added to the test dataset.

$$\boldsymbol{x}_t = \boldsymbol{x}_t + [\,1\ 1\ 1\ 1\ 1\ 1\,]^T \tag{14}$$

100 pieces of data are generated and added to the test dataset.

Set the epoch of pre-training iterations of autoencoder training as 500 and the epoch of joint training as 500. The reconstruction error curve of the test samples is shown in Fig 4.

The probability density of reconstruction error of normal samples was estimated by KDE, and the control limit of reconstruction error value is set as 16.59 according to the criterion that 95% of normal samples are lower than the control limit. According to the classification results in Fig 5, the proposed model can detect fault samples.

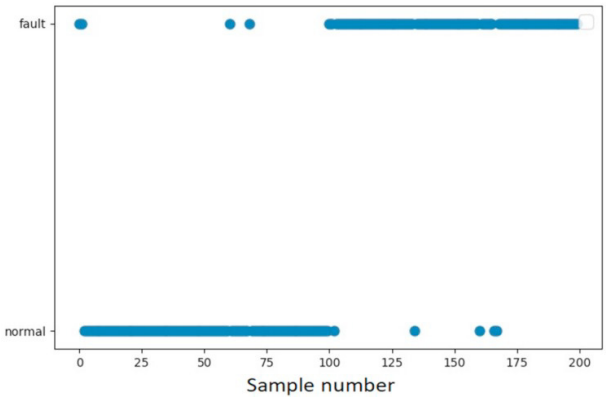


Fig. 5. Classification results of simulation data.

4.2 The Steel Plate Production Process

Steel plate production belongs to process industries,which is a continuous process. Steel plate production is divided into heating, rolling, cooling and straightening . Five variables in the rolling process which are easy to cause unqualified plate shape were selected for the experiment, and the physical meanings of these variables were: the amount of roll down, torque of the lower roll, torque of the upper roll, the contact length between the roll and the steel plate, and flat shape at the exit. In the training set, there were 2000 normal samples and 100 fault samples with the minority being 5% of the majority. In the testing set, balanced data were selected, with 100 normal samples and 100 fault samples respectively. Since we used time-series data, a sample represents the dynamic change process of a steel plate, and each sample contains 5 features and 100 time steps.

All normal samples of training are used for model training. The epoch of autoencoder training and joint training are set to 500, respectively. Then the testing set is input into the model for anomaly detection, and the reconstruction error of the testing set is shown in Fig 7:

In order to determine the control limit of the reconstruction error, kernel density estimation was used to estimate the probability density of the normal samples in the testing set. According to the criterion that 95% of the normal samples are lower than the control limit, the control limit was set as 51.00.

To demonstrate the superiority of our proposed model, we compared it with supervised algorithms, unsupervised algorithms and anomaly detection models based on data enhancement. We selected wasserstein GAN (WGAN) (Arjovsky et al. (2017)) with good generating ability as the data enhancement method, expanded fault samples, generated 1900 synthetic samples to balance the training data, and then input them into LDA for anomaly detection. We also added a comparison experiment with the basic AE model based on GRU, and the epoch of the AE model is set to 1000 under the same conditions for the experiment.In order to better highlight the superiority of the proposed model, F1 score is introduced as a comprehensive evaluation index.

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall} \tag{15}$$

The classification results are shown in Fig 6, and the result of each of methods is shown in Table 1.

Table 1. Comparison of the main metric results for each of the methods

Method	Prediction			
	Precision	Recall	F1	Accuracy
KNN	0.9836	0.6	0.7453	0.795
LDA	1.0	0.73	0.8439	0.865
PCA	0.9359	0.73	0.8202	0.84
OCSVM	0.3896	0.3	0.339	0.415
AE	0.8235	0.7	0.7567	0.775
WGAN+LDA	0.9746	0.77	0.8603	0.875
The proposed method	0.9307	0.94	**0.9353**	**0.935**

It can be seen from the classification results that our proposed method is undoubtedly the best. Supervised algorithms such as KNN and LDA have low classification accuracy due to the extremely imbalanced data. Unsupervised algorithms such as PCA and OCSVM do not consider the dynamic relationship between data, so the detection accuracy is reduced. Since WGAN cannot learn the dynamic features between time-series data, the quality of data generated by WGAN is reduced, and the classification accuracy is only slightly improved compared with that of LDA without data enhancement. The loss function of the self-coding network in the standard AE and the proposed method is shown in Fig 8. It can be seen that in our method, the value of the loss function is smaller, and the AE network is better trained.

5. DISCUSSION AND FUTURE WORK

In this paper, anomaly detection for multi-dimensional time series is studied in the case of extreme lack of fault samples, and an autoencoder anomaly detection model based on generative adduction network assisted training is proposed. The proposed model creatively introduces GAN into the training of autoencoder, and greatly improves the anomaly detection ability of the autoencoder by judging the similarity between the first embedding and the second embedding. We not only verified the validity of the proposed model in simulation data, but also applied it to the anomaly detection in steel plate production process. Since we set the control limit to 95%, the precision of the proposed method is around 0.95. Compared with the anomaly detection methods based on statistical learning and data enhancement, the classification recall, accuracy and F1 score of the proposed model are the best. In future work, we will further reduce reconstruction error of training samples according to the features of the data, so as to make the boundary between normal samples and fault samples more obvious and the classification accuracy higher in anomaly detection.

ACKNOWLEDGEMENT

This work was supported by the National Key Research and Development Project (2020YFB1710003).

REFERENCES

Abeywickrama, T., Cheema, M.A., and Taniar, D. (2016). K-nearest neighbors on road networks: a journey in experimentation and in-memory implementation. *arXiv preprint arXiv:1601.01549.*

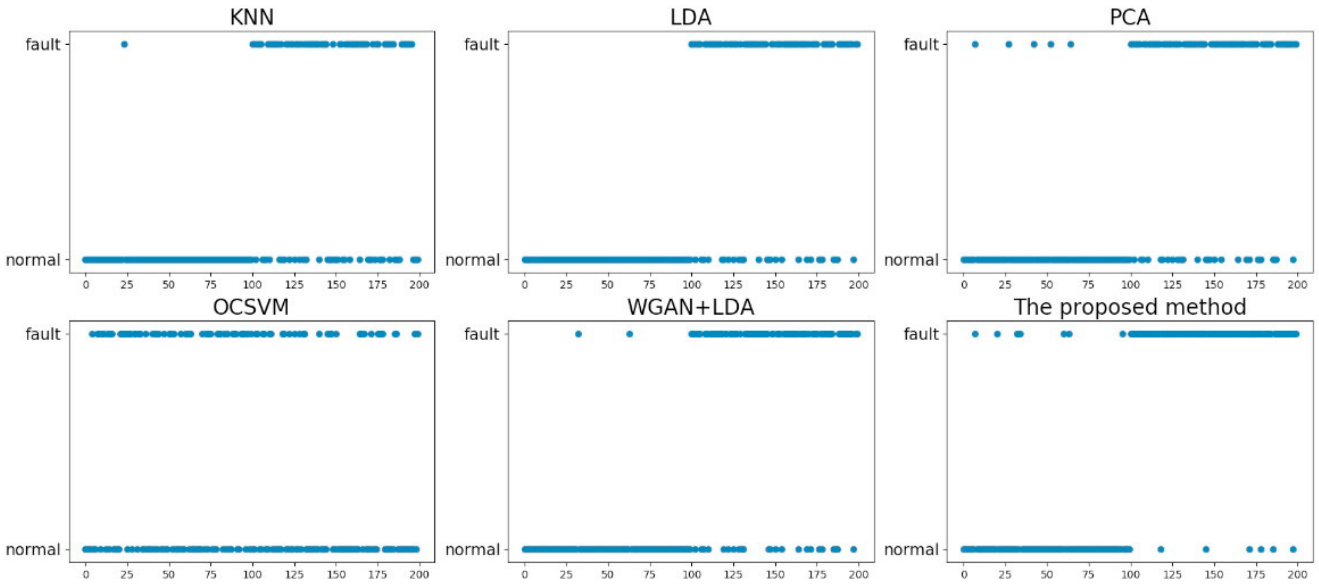


Fig. 6. Classification results on the steel plate production process.

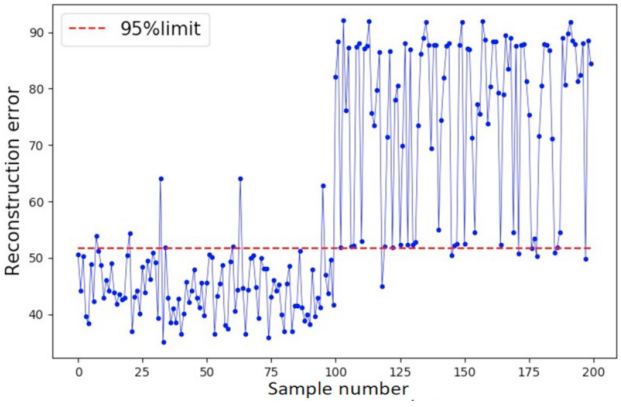


Fig. 7. The reconstruction error curve of the proposed method.

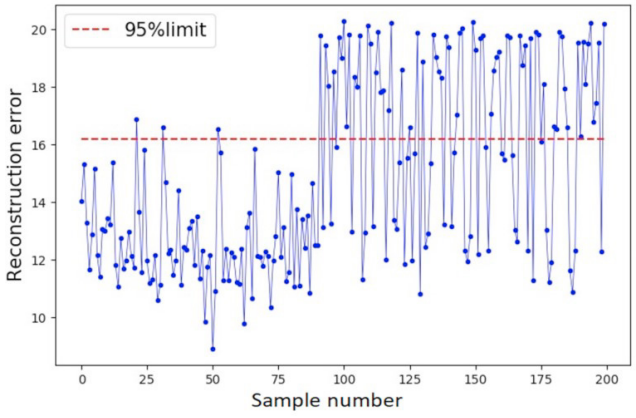


Fig. 8. The reconstruction error curve of autoencoder.

Arjovsky, M., Chintala, S., and Bottou, L. (2017). Wasserstein generative adversarial networks. *International Conference on Machine Learning*, 214–223. PMLR.

Bansal, T., Belanger, D., and McCallum, A. (2016). Ask the gru: Multi-task learning for deep text recommendations. 107–114.

Cheng, Z., Zou, C., and Dong, J. (2019). Outlier detection using isolation forest and local outlier factor. *Proceedings of the Conference on Research in Adaptive and Convergent Systems*, 161–168.

Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial networks (2014). *arXiv preprint arXiv:1406.2661*, 1406.

Schölkopf, B., Platt, J.C., Shawe-Taylor, J., Smola, A.J., and Williamson, R.C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7), 1443–1471.

Shlens, J. (2014). A tutorial on principal component analysis. *arXiv preprint arXiv:1404.1100*.

Tang, Y. (2013). Deep learning using linear support vector machines. *arXiv preprint arXiv:1306.0239*.

Tian, Y., Xu, Y., Zhu, Q.X., and He, Y.L. (2022). Novel stacked input-enhanced supervised autoencoder integrated with gated recurrent unit for soft sensing. *IEEE Transactions on Instrumentation and Measurement*, 71, 1–9.

Yin, S., Ding, S.X., Xie, X., and Luo, H. (2014). A review on basic data-driven approaches for industrial process monitoring. *IEEE Transactions on Industrial Electronics*, 61, 6418–6428.

Yoon, J., Jarrett, D., and Van der Schaar, M. (2019). Time-series generative adversarial networks. *Advances in Neural Information Processing Systems*, 32.

Zhang, C., Liu, Y., and Fu, H. (2019). Ae2-nets: Autoencoder in autoencoder networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2577–2585.

Zhu, F., Gao, J., Yang, J., and Ye, N. (2022). Neighborhood linear discriminant analysis. *Pattern Recognition*, 123, 108422.